

TP 5

Tableaux et matrices

1. A quoi ça sert ?

Nous venons de voir en cours ce qu'est une matrice. Scilab est extrêmement bien adapté à la manipulation de matrices (ou de tableaux). En fait, c'est même la structure de données principale de Scilab.

On a déjà pu se rendre compte de cela avec la fonction **grand(n,m, 'uin',a, b)** qui renvoie une matrice de taille (n,m) de lancers de dé à 6 faces. Cet exemple est d'ailleurs une très bonne illustration de l'intérêt informatique de stocker des données sous forme de matrice. En effet, quel objet mathématique aurait été plus pratique pour retourner cette *liste* de résultats de lancers de dés ?

Dire que Scilab est parfaitement adapté à la manipulation des matrices, c'est dire que les commandes usuelles de Scilab sont très intuitives sur les matrices. On a déjà vu constater cela lorsqu'on a manipulé par exemple la fonction **sum** sur un vecteur y . Scilab comprend qu'on souhaite qu'il fasse la somme des coordonnées de ce vecteur. En fait, la plupart des commandes liées aux tableaux ou aux matrices sous Scilab sont aussi instinctives que celle-là.

Enfin, Scilab permet de construire toutes sortes de matrices, à partir des réels, mais aussi en utilisant des booléens, des chaînes de caractères... on peut ensuite effectuer toutes les opérations algébriques usuelles définies sur les matrices.

L'objectif de ce TP est de vous faire découvrir comment manipuler les matrices sous Scilab. Je vous conseille de vous faire une fiche de tout ce que vous avez appris au cours de ce TP pour être capable de réutiliser les commandes facilement.

2. Comment créer et modifier une matrice sous Scilab ?

Afin de découvrir la syntaxe pour créer une matrice sous Scilab, taper les instructions suivantes directement dans la console.

```
--> [1, 2, 3, 4 , 5, 6, 7, 8]
--> [1 ;2; 3; 4; 5; 6; 7; 8]
--> [1, 2, 3, 4; 5, 6, 7, 8]
```

Avez-vous compris la différence entre la virgule et le point-virgule ?

A la place des virgules, on peut se contenter d'écrire un espace

Exercice 1. Créer les matrices suivantes sous Scilab

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 1 & 8 \end{pmatrix}$$

3. Matrices particulières

Matrices régulières :

Lorsque l'on travaille avec des matrices de taille élevée, il peut être bien pénible de rentrer tous les coefficients à la main. Dans certains cas particuliers, on peut définir les matrices plus rapidement. Essayer par exemple de taper les commandes suivantes dans la console.

```
--> 1:6
--> 2:28
--> 1:2:8
--> 10:-1:1
```

Que représente le nombre de droite ? Celui du milieu ? Et celui de gauche ?

Exercice 2. Utiliser cette technique pour créer le tableau des entiers pairs entre 0 et 100 rangés par ordre croissant ($[0; 2; 4; 6; 8; \dots; 100]$), puis celui des mêmes entiers par ordre décroissant ($[100; 98; 96; \dots; 0]$).

La fonction `linspace(x,y,n)` permet de créer un tableau contenant n valeurs **réparties régulièrement** de x à y . L'essayer en tapant ces commandes dans la console :

```
--> L=linspace(1,10,10)
--> L=linspace(1,10,1)
--> L=linspace(1,10,100)
```

On notera, au passage, qu'il est possible de **concaténer** des matrices pour en créer de nouvelles. Essayez :

```
--> L1=[1:5]
--> L2=[7:2:15]
--> [L1, L2]// on place L1 à gauche de L2
--> [L1; L2]// on place L1 au-dessus de L2
```

Mais il faut faire attention aux tailles des matrices !

Matrices classiques :

On retiendra enfin les commandes suivantes permettant de définir les matrices **classiques** :

1. La **matrice nulle** de taille (n, p) se crée à l'aide de la commande `zeros(n,p)`.
2. De même, la **matrice composée uniquement de 1** se crée à l'aide de la commande `ones(n,p)`.
3. La **matrice identité** de taille n se crée à l'aide de la commande `eye(n,n)`.
4. La commande `diag(v)`, où v est une matrice (ligne ou colonne) permet de construire une **matrice diagonale** dont la diagonale est v . Inversement, la commande `diag(A)`, où A est une matrice (pas nécessairement diagonale) permet de construire une matrice colonne dont les composantes sont celles de la diagonale de la matrice.

Enfin, en tapant `diag(v,1)`, on construit une matrice diagonale de taille adaptée en conséquence dont la première sur-diagonale est v (on aussi peut taper `diag(v,2)` pour la deuxième sur-diagonale, `diag(v,-1)` pour la première sous-diagonale, etc...

Exercice 3. 1. Construire $C = 0_4$.

2. Construire une matrice D à 4 lignes et 3 colonnes composée uniquement de 1.

3. Essayer les commandes suivantes : `eye(3,3)`, `eye(2,3)`, `eye(3,2)`.
4. Créer une matrice carrée de taille 3 contenant 1, 2, 3 sur la diagonale.

Matrices aléatoires :

1. On a déjà vu que `grand(n,p,'uin',a,b)` génère une matrice de taille (n,p) remplie de tirages indépendants de la loi uniforme sur $\llbracket a;b \rrbracket$ (c'est à dire un lancer de dé à $b-a+1$ faces). En fait, la fonction `grand` est très puissante sous Scilab. En changeant le préfixe `'uin'`, on peut obtenir des tirages de n'importe quelle loi de probabilité. Nous y reviendrons en détail plus tard.
2. De la même manière, la commande `rand(n,p)` permet de générer une matrice contenant des **nombre aléatoire**, chacun étant tiré au hasard indépendamment des autres selon une loi **uniforme** dans l'intervalle $[0; 1[$. C'est fois, on ne considère plus que les entiers mais bien tous les nombres entre 0 et 1.

Exercice 4. Simuler à l'aide d'une seule matrice H une série de 20 expériences aléatoires similaires, chacune de ces expériences consistant à lancer 100 fois un dé (équilibré), le résultat d'une expérience correspondant à une colonne de H .

4. Opérations sur les matrices

Accéder à des éléments d'une matrice

Lorsqu'on manipule des matrices, il est intéressant de pouvoir accéder à une ligne, un coefficient, ... pour les isoler ou même pour les changer. Taper les commandes suivantes, en essayant préalablement de trouver ce qu'elles vont faire.

```
--> A=1:2:10;           --> B=[1, 2, 3, 4; 5, 6, 7, 8]
--> A(1)                --> B(2,3)
--> A(4)                --> B(2,3)=4
--> A(4)=25             --> B
--> A(6)=61             --> B(2,:)
--> A                   --> B(:,1)
--> A(3)=A(4)           --> B(1,:)= [1,1,1,1]
```

- Exercice 5.**
1. Entrer une matrice C de taille $(3,4)$ de votre choix. Remplacer la deuxième ligne de la matrice C par une ligne de 1, puis remplacer la deuxième colonne de la matrice C par la première colonne de la matrice C .
 2. Echanger les colonnes 1 et 2 de la matrice C .

Opérations élémentaires sur les matrices

Si, comme nous l'avons vu en introduction, les commandes liées aux opérations élémentaires sur les matrices sont assez instinctives, il faut cependant faire attention à bien distinguer deux types d'opérations mathématiquement différentes dans certains cas :

- les opérations « **coefficient par coefficient** ».
- les **opérations matricielles** classiques.

Pour la somme, ces opérations sont les mêmes puisque que sommer deux matrices, c'est sommer chaque coefficients des deux matrices entre-eux. En revanche, si on travaille avec le produit, ce n'est pas la même opération que de faire le produit de chaque coefficient des deux matrices ou de faire le produit matriciel des deux matrices. Scilab est capable de faire les deux, il suffit de lui donner la bonne commande et de faire attention aux compatibilités de dimension.

Commençons par étudier les opérations matricielles comme elles sont définies dans le cours de maths.

Créer les matrices suivantes sous Scilab

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 2 & 5 \\ 2 & 6 & 1 \end{pmatrix} \quad C = \begin{pmatrix} 4 & 2 & 2 \\ 1 & 3 & 1 \\ 2 & 6 & 0 \end{pmatrix}$$

Taper les commandes suivantes

```
--> A+B
--> A*B
--> B*A
--> A^10
--> B^10
```

$B + A$, $A * B$, $B * A$, A^{10} et B^{10} en anticipant les messages d'erreur et comprenant ce qu'elles font.

Voyons maintenant comment effectuer des opérations coefficient par coefficient.

Taper les commandes suivantes et comprendre ce qu'elles permettent de faire. (On utilise les matrices A et B précédentes).

```
--> A+2
--> A.*C
--> A./C
--> A.^3
```

Enfin, on notera qu'on peut appliquer n'importe quelle fonction prédéfinie de Scilab à une matrice, le calcul d'image s'effectuera coefficient par coefficient. Essayer et commenter par exemple

```
--> sqrt(A)
--> exp(A)
--> sum(A)
--> prod(A)
--> max(A)
--> min(A)
```

Inverse d'une matrice

L'**inverse** d'une matrice M se calcule via la commande `inv(M)` qui laisse un message d'erreur lorsque la matrice n'est pas inversible. *Attention toutefois, il faut se rappeler que Scilab ne fait que des calculs approchés, ce qui donne parfois des erreurs de la part du logiciel.*

Exercice 6. 1. Donner l'inverse de $C = \begin{pmatrix} 3 & 1 & 4 \\ 2 & 5 & 7 \\ 1 & 0 & 0 \end{pmatrix}$, puis calculer $C^{(-1)} * C$ et $1/C$ et commenter.

Calculer $1./C$ et $(1) ./C$ et commenter.

2. Effectuer `inv(B)`, `inv(A)` et commenter.

Matrice transposée

La **transposée** d'une matrice M est obtenue à partir de la commande `M'`

Exercice 7. 1. Vérifier l'instruction en effectuant A' .

2. A partir de la matrice ligne T contenant les entiers de 1 à 10, créer une matrice de taille 10 qui contient les résultats des tables de multiplication.

Indication : penser à la transposée de T

Taille d'une matrice

Lorsqu'on considère une matrice **A** de taille inconnue on peut récupérer le nombre de lignes et/ou de colonnes par les instructions **size(A)**, **size(A,1)**, **size(A,2)**. On peut aussi utiliser **length()** quand la matrice a une seule ligne ou une seule colonne pour récupérer sa longueur. Essayer et comprendre les commandes suivantes :

```
--> size(B)
--> size(B,1)
--> size(B,2)
--> y=grand(1,n,'uin', 1,6);
--> length(y)
```

5. A vous de jouer !

- Exercice 8.**
1. Écrire une fonction **matprod** qui effectue le produit de deux matrices lorsque c'est possible et vous signale l'erreur dans le cas contraire.
 2. Écrire une fonction **mattrace** qui calcule la trace d'une matrice carrée, c'est-à-dire la somme des coefficients diagonaux de cette matrice.
 3. Écrire une fonction **danstab** qui teste si un élément **x** appartient à une matrice **A**.

Exercice 9. Créer une matrice qui donne les sommes des points de deux dés, l'indice de ligne (1 à 6) représentant le résultat du dé 1 et l'indice de colonne (1 à 6) représentant le résultat du dé 2. En déduire le dénombrement des situations conduisant aux différentes sommes de 2 à 12.

Exercice 10. Soit **A** la matrice définie par : $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$. Calculer A^n pour tout $n \in \mathbb{N}$.

On pourra conjecturer la forme de A^n à l'aide de Scilab.

Exercice 11. Créer la matrice $n \times n$ suivante :

$$\begin{pmatrix} 1 & 1 & 0 & \dots & 0 \\ 1 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 1 & 0 & 0 & \ddots & 1 \\ 1 & 0 & 0 & \dots & 0 \end{pmatrix}.$$

Exercice 12. Quelle matrice est créée par cette fonction ?

```
function y=mystere(n)
P=zeros(n,n);
for i=1:n
    P(i,i)=1;
    P(i,1)=1;
end
    for i=3:n
        for j=2:n
            P(i,j)=P(i-1,j-1)+P(i-1,j)
        end
    end
end
y=P
endfunction
```